

HW10

Sam Ly

April 18, 2026

Chapter 20

<https://github.com/samlyme/Assignments/tree/cs4080/ch20/main/CS4080>

1. Add support for keys of the other primitive types: numbers, Booleans, and nil. Later, clox will support user-defined classes. If we want to support keys that are instances of those classes, what kind of complexity does that add?

First, we create a general ObjHashable, so we no longer hardcode in ObjString as our default hash key. Then, we use the struct inheritance trick covered earlier.

```
struct Obj {
    ObjType type;
    struct Obj* next;
};

struct ObjHashable {
    Obj obj;
    void* data;
    size_t length;
    uint32_t hash;
};

struct ObjString {
    ObjHashable objHashable;
    int length;
    char* chars;
};
```

The rest is just fixing type casts or struct member names.

2. Look up a few hash table implementations in different open source systems, research the choices they made, and try to figure out why they did things that way.

Rust's standard library has a hashmap implementation that uses the Swisstable algorithm. First, there is always a power of 2 capacity. This is because modulo a power of 2 can be optimized away as a bit manipulation by compilers. Then, we use the similar EMPTY, DELTED(tombstone), and FULL states, but we explicitly encode them as a "control byte" in the entry type, rather than how we do it, with a null pointer. This is just a choice they made.

For the actual algorithm, the key difference is that they implement SIMD byte matching. That means large chunks of the open-addressed table are searched in parallel. This means that, most of the time, only one "walk" is needed to find your key. We can also do this for ours, but requires some extra work.

Here is the key implementation difference: probing is done with larger steps. Specifically, each "probe", we jump by 1 chunk, then 2, then 3. This leaves larger gaps in our table, which is good, since it increases uniformity of the distribution of hashed values. Also, this is still guaranteed to terminate if there is an empty bucket for some mathematical reason.

This is the main design choice difference, and it is pretty interesting. There are some other minute details and edge cases to take care of, but this is the crux of the implementation.

3. Write a handful of different benchmark programs to validate our hash table implementation. How does the performance vary between them? Why did you choose the specific test cases you chose?

```
void benchmarkTable(const char *filePath) {
    FILE* file = fopen(filePath, "r");
    if (file == NULL) {
        fprintf(stderr, "Could not open file '%s'.\n", filePath);
        return;
    }

    // each line is <key>:<value>, assume is just string for now.
    Table table;
    initTable(&table);

    char line[1024];
    char key[128];
    char value[128];
    size_t totalLines = 0;
    size_t validEntries = 0;
    size_t invalidLines = 0;
    size_t bytesRead = 0;
    clock_t start = clock();

    while (fgets(line, sizeof(line), file)) {
        totalLines++;
        bytesRead += strlen(line);
        line[strcspn(line, "\n")] = '\0';

        if (sscanf(line, "%127[^:]:%127s", key, value) == 2) {
            ObjString* k = copyString(key, (int)strlen(key));
            Value v = OBJ_VAL(copyString(value, (int)strlen(value)));

            tableSet(&table, k, v);
            validEntries++;
        } else {
            fprintf(stderr, "Invalid line: %s\n", line);
            invalidLines++;
        }
    }

    clock_t end = clock();
    double elapsedSeconds = (double)(end - start) / CLOCKS_PER_SEC;
    double entriesPerSecond =
        elapsedSeconds > 0.0 ? validEntries / elapsedSeconds : 0.0;
    double megabytesPerSecond =
        elapsedSeconds > 0.0 ? (bytesRead / (1024.0 * 1024.0)) / elapsedSeconds
        : 0.0;
}
```

```

fclose(file);

printf("Benchmark: %s\n", filePath);
printf("  lines read: %zu\n", totalLines);
printf("  valid entries: %zu\n", validEntries);
printf("  invalid lines: %zu\n", invalidLines);
printf("  bytes read: %zu\n", bytesRead);
printf("  table count: %d\n", table.count);
printf("  table capacity: %d\n", table.capacity);
printf("  elapsed: %.6f s\n", elapsedSeconds);
printf("  throughput: %.2f entries/s, %.2f MiB/s\n",
       entriesPerSecond, megabytesPerSecond);

freeTable(&table);
}

```

Overall, pretty good.

Chapter 21

1. The compiler adds a global variable's name to the constant table as a string every time an identifier is encountered. It creates a new constant each time, even if that variable name is already in a previous slot in the constant table. That's wasteful in cases where the same variable is referenced multiple times by the same function. That, in turn, increases the odds of filling up the constant table and running out of slots since we allow only 256 constants in a single chunk.

Optimize this. How does your optimization affect the performance of the compiler compared to the runtime? Is this the right trade-off?

A simple optimization is to iterate through the constants table to see if an identifier has already been added there. If it has, we don't need to add a new entry to the constants table. This is alright, but the issue is that this introduces a $O(n)$ operation to every single variable usage, which scales poorly. Although we are technically in "constant time" because we know there can only be up to 256 values, this is still a significant issue. Luckily, due to caching and branch prediction, the performance is likely not to be terrible. However, the upside does outweigh the down, because this fixes a very real issue in our runtime: printing too many values will result in an OOM like error, as we will fill up the constant table, so this tradeoff is worth it.

```

static uint8_t makeConstant(Value value) {
    Chunk* chunk = currentChunk();

    int constantIdx = -1;

    for (int i = 0; i < chunk->constants.count; i++) {
        printf("look for constant: i: %d\n", i);
        if (valuesEqual(chunk->constants.values[i], value)) {
            constantIdx = i;
            printf("~~~~~ seen variable before!!\n");
            break;
        }
    }
    if (constantIdx == -1) constantIdx = addConstant(chunk, value);
}

```

```

    if (constantIdx > UINT8_MAX) {
        error("Too many constants in one chunk.");
        return 0;
    }

    return (uint8_t)constantIdx;
}

```

2. Looking up a global variable by name in a hash table each time it is used is pretty slow, even with a good hash table. Can you come up with a more efficient way to store and access global variables without changing the semantics?

One way to improve this is to do the hashing and whatnot at the compile stage, and translate all identifiers into raw constant table indices. That way, the VM won't need to read a string as the variable name and do the hash table lookup.

3. We could report mistakes like this as compile errors, at least when running from a script. Do you think we should? Justify your answer. What do other scripting languages you know do?

I would not make this a compile error (which does make the prior optimization) harder), because we have already established that the global namespace is dynamic, and evolves as the program runs. Although in this case we *can* tell that this will result in an error statically, this may not be the case in all situations, and it would be best to leave this as a runtime error. Most other scripting languages like Python or JS also do this.